

Trabajo Práctico 2 — Tu primer juego de texto en GDScript

Diplomatura de Videojuegos · Clase 2 Objetivo: practicar **variables**, **arreglos**, **if**, **for**, **funciones** e **input** escribiendo código **de verdad**, todo probado **en la consola** (`print`). Vas a construir, paso a paso, un pequeño **combate por turnos**: *La Cripta del Golem*.

Qué vas a lograr

Un mini juego **100% de consola** (sin sprites todavía): un duelo por turnos donde elegís arma, atacás y tomás pociones, y todo se ve en el panel **Output** de Godot.

```
=== LA CRIPTA DEL GOLEM ===
Aria se enfrenta a un Golem de piedra!
Tu arsenal:
[0] Espada (daño 25)
[1] Arco (daño 15)
[2] Hacha (daño 40)
Controles: ← →  cambiar arma | ENTER  atacar | ↑  poción
---
Aria: 100 HP | Golem de piedra: 120 HP | Pociones: 2
---
```

Vas a llegar ahí **sumando de a poco**: primero probás cada concepto solo (variables, `if`, `for` ...) y al final los juntás en el juego.

 **Tiempo estimado:** 60–90 min. La clave: **escribí el código vos** (no copies y pegues todo). Se aprende tecleando y rompiendo.

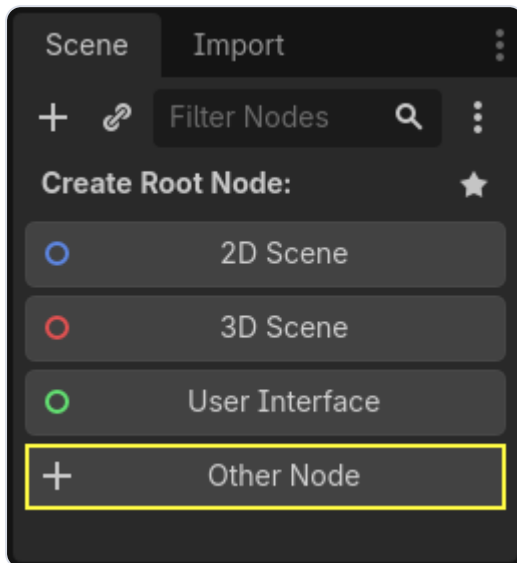
Antes de arrancar: ¿por qué “en la consola”?

En la clase vimos que `print()` es nuestro mejor amigo para depurar. Hoy lo usamos como *pantalla*: en vez de mover un personaje, vamos a **imprimir** lo que pasa. Así nos concentramos en la **lógica** (variables, decisiones, repeticiones) sin pelear con gráficos.

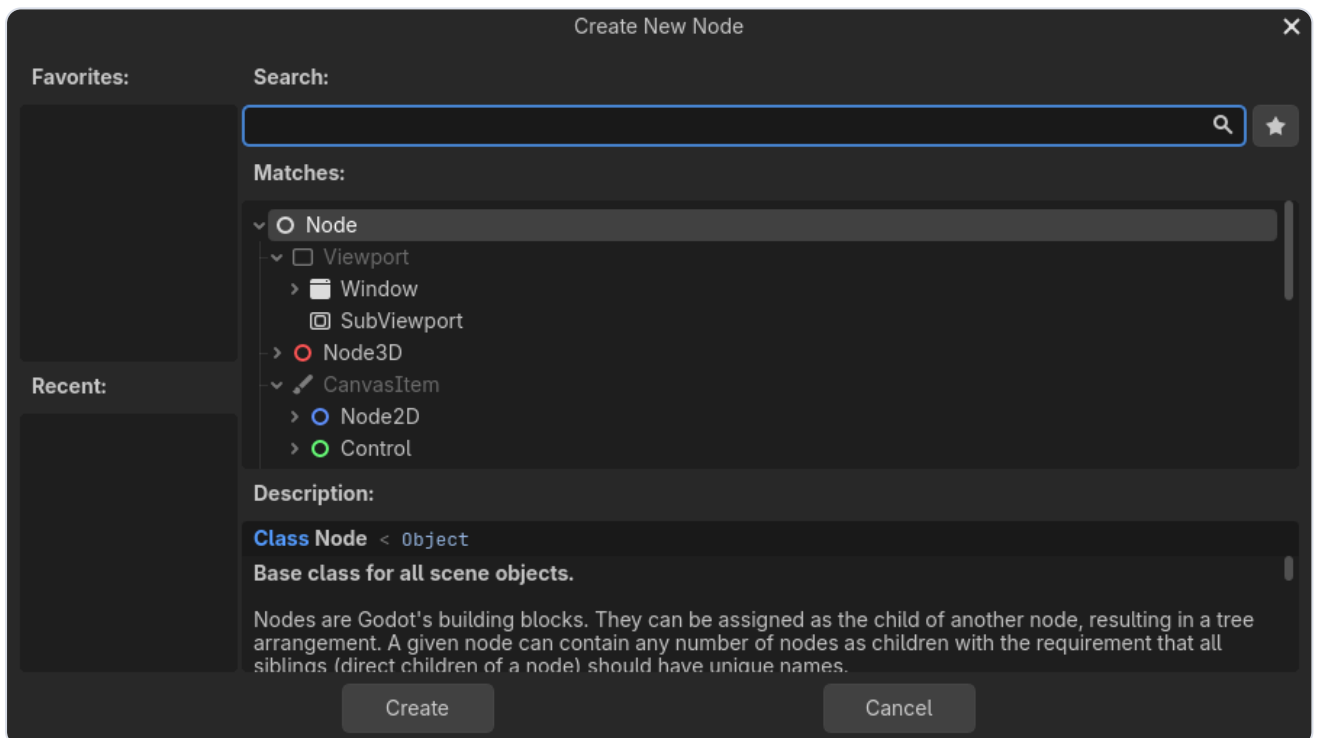
Lo único gráfico va a ser el **input**: leemos las teclas con el sistema `Input` de Godot (como en clase), pero la “imagen” del juego es el texto que aparece en el panel **Output**.

Parte 0 — Proyecto, nodo y script

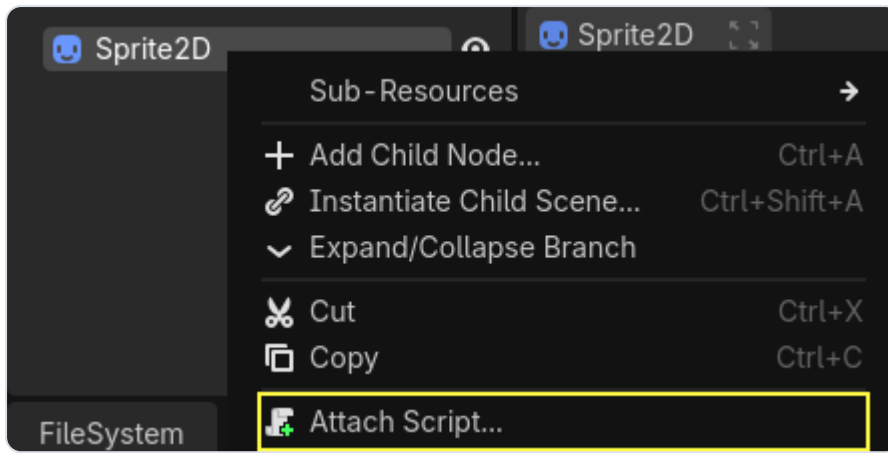
1. Abrí Godot → **New Project** → nombre `tp2-crypta` → carpeta vacía → **Create & Edit**.
2. En el panel **Escena**, clic en **Otro Nodo** (*Other Node*).



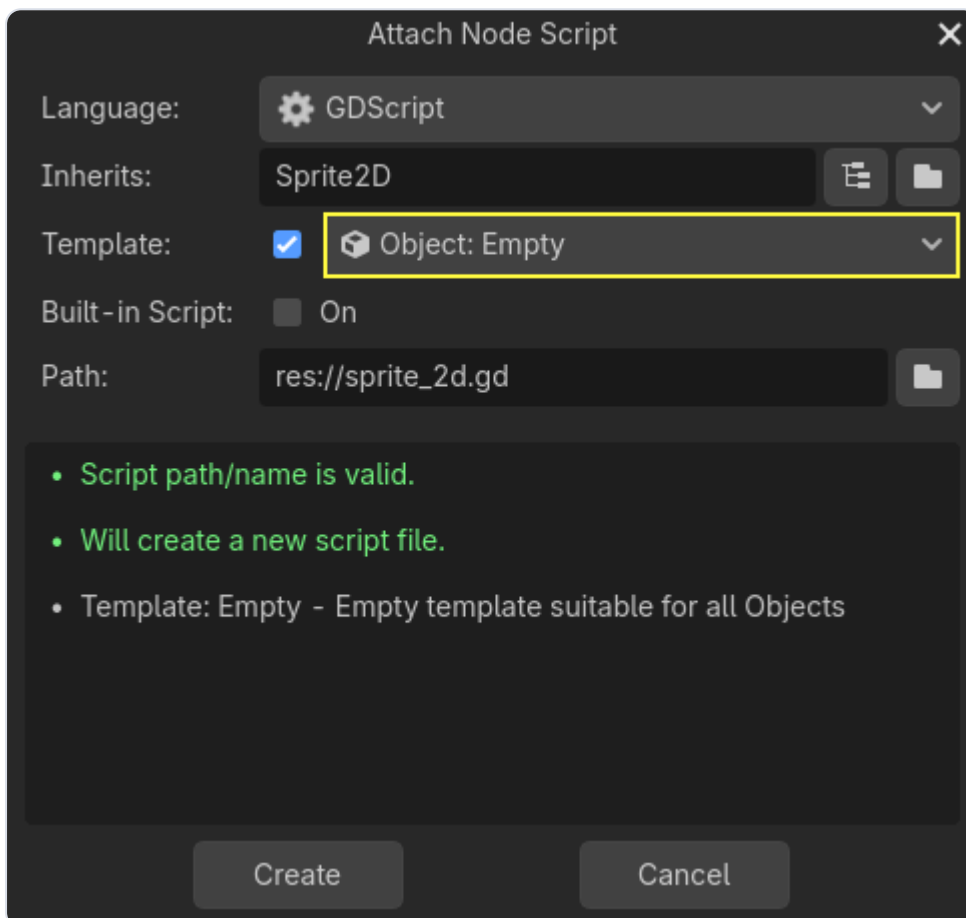
3. Buscá `Node` (el nodo más básico, sin nada visual), selecciónalo y **Crear**.



4. Renombralo a `Juego` (doble clic sobre el nombre).
5. **Adjuntá un script**: clic derecho sobre `Juego` → **Attach Script** (*Adjuntar Script*).



6. En el diálogo, dejá **Language: GDScript** y **Path** en `res://juego.gd`. Clic en **Create**.



7. Se abre el **editor de Script**. Godot te deja una plantilla. **Borrá todo** y dejá solo esto:

```
extends Node

func _ready():
    print("¡El juego arrancó!")
```

8. Guardá con **Ctrl + S** y **ejecutá la escena** con **F6**. Si te pregunta, guardá la escena como `juego.tscn`.
9. Mirá el panel inferior **Output** (*Salida*): tiene que aparecer tu mensaje.

```
Godot Engine v4.6.rc1.official
D3D12 12_0 - Forward+ - Using
1060 6GB

Hello, world!
--- Debugging process stopped
```

 `extends Node` dice qué tipo de nodo controla el script. `func _ready():` se ejecuta **una sola vez** al aparecer el nodo. Todo lo que imprimas ahí aparece en **Output**.

✅ **Punto de control 0:** ves "¡El juego arrancó!" en el panel Output al apretar F6.

❌ **No aparece nada / da error**

▼ Abrió para ver soluciones

- La **indentación importa**: lo de adentro de `func` va con **una sangría** (4 espacios o un tab, pero **no mezcles** los dos).
- ¿Guardaste con **Ctrl + S** antes de apretar F6?
- ¿El panel de abajo está en la pestaña **Output**, no en **Debugger**?

Parte 1 — Variables: la ficha del héroe

Concepto: asignación de variables (`var`) y tipos (`int` , `float` , `String` , `bool`), imprimir con `str()` .

Reemplazá el `func _ready()` por esto y apretá **F6**:

```
extends Node

func _ready():
    var nombre = "Aria"           # String  (texto)
    var vida = 100                 # int    (entero)
    var precision = 85.5           # float  (decimal)
    var es_heroe = true            # bool   (verdadero/falso)

    print("Nombre: " + nombre)
    print("Vida: " + str(vida))
    print("Precisión: " + str(precision) + "%")
    print("¿Es heroína?: " + str(es_heroe))
```

Deberías ver en Output:

Nombre: Aria
Vida: 100
Precisión: 85.5%
¿Es heroína?: true

🗨️ Una variable es una **caja con etiqueta**: `vida` guarda un `100`. Para **pegar** un número con texto hay que convertirlo con `str()` — `"Vida: " + str(vida)`. Si te olvidás el `str()`, Godot se queja: no sabe sumar texto con número.

🔧 **Probá vos:** cambiá los valores. Poné tu propio nombre y una vida distinta. Volvé a apretar F6 y mirá cómo cambia la salida.

✅ **Punto de control 1:** imprimís una ficha con los 4 tipos de datos.

📦 Parte 2 — Arreglos: el arsenal

Concepto: arreglos (arrays) — crear, acceder por **índice**, tamaño con `.size()`, agregar con `.append()`.

Un **arreglo** es una variable que guarda **varias cosas en orden**. En la clase vimos `["Espada", "Escudo", "Poción"]`. Cada elemento tiene un **número de posición** (índice) que **empieza en 0**.

```
extends Node

func _ready():
    var armas = ["Espada", "Arco", "Hacha"]

    print("Primer arma: " + armas[0])      # índice 0 → Espada
    print("Segunda arma: " + armas[1])     # índice 1 → Arco
    print("Cuántas armas tengo: " + str(armas.size()))

    armas.append("Daga")                  # agrega al final
    print("Nueva arma agregada: " + armas[3])
    print("Ahora tengo: " + str(armas.size()) + " armas")
```

Output:

```
Primer arma: Espada
Segunda arma: Arco
Cuántas armas tengo: 3
Nueva arma agregada: Daga
Ahora tengo: 4 armas
```

🗨️ **El índice empieza en 0.** En un arreglo de 3 elementos, las posiciones son `0`, `1` y `2`. Pedir `armas[3]` **antes** de agregar la Daga daría error ("índice fuera de rango"): esa posición todavía no existe.

💡 **Truco de diseño — arreglos en paralelo:** guardamos el **daño** de cada arma en **otro** arreglo, en el mismo orden. `armas[2]` es `"Hacha"` y `danos[2]` es su daño. Los vamos a usar así en el juego final.

```
var armas = ["Espada", "Arco", "Hacha"]
var danos = [25, 15, 40]
print(armas[2] + " hace " + str(danos[2]) + " de daño") # Hacha hace 40 de daño
```

✅ **Punto de control 2:** accedés a elementos por índice y usás `.size()` y `.append()`.

🔗 Parte 3 — Condicionales: decidir el resultado

Concepto: `if` / `elif` / `else` y operadores de comparación (`==`, `!=`, `<`, `>`, `<=`, `>=`).

```
extends Node

func _ready():
    var vida_enemigo = 40
    var dano = 25

    vida_enemigo = vida_enemigo - dano
    print("Vida del enemigo: " + str(vida_enemigo))

    if vida_enemigo <= 0:
        print("💀 El enemigo cayó")
    elif vida_enemigo <= 20:
        print("🔴 El enemigo está por morir")
    else:
        print("🟢 El enemigo sigue fuerte")
```

Output (con estos valores):

```
Vida del enemigo: 15
🔴 El enemigo está por morir
```

🗨️ Godot revisa las condiciones **de arriba hacia abajo** y ejecuta **la primera que sea verdadera**. Con `vida_enemigo = 15`: no es `<= 0`, **sí** es `<= 20` → imprime la de vida crítica y **saltea el resto**.

🔗 **Probá vos:** cambiá `dano` a `50` (¿qué rama sale?) y a `10` (¿y ahora?). Predecí el resultado **antes** de apretar F6.

✅ **Punto de control 3:** según el daño, cae en una rama distinta.

🔄 Parte 4 — `for`: repetir sin copiar y pegar

Concepto: el loop `for`, con `range()` y recorriendo un arreglo.

```
extends Node

func _ready():
    # 1) Repetir N veces con range()
    print("--- Aparecen enemigos ---")
    for i in range(3):
        print("Golem #" + str(i + 1) + " entró a la cripta")

    # 2) Recorrer un arreglo elemento por elemento
    print("--- Tu arsenal ---")
    var armas = ["Espada", "Arco", "Hacha"]
    for arma in armas:
        print("Tenés: " + arma)

    # 3) Recorrer con el índice (para arreglos en paralelo)
    var danos = [25, 15, 40]
    for i in range(armas.size()):
        print "[" + str(i) + "] " + armas[i] + " → daño " + str(danos[i]))
```

Output:

```
--- Aparecen enemigos ---
Golem #1 entró a la cripta
Golem #2 entró a la cripta
Golem #3 entró a la cripta
--- Tu arsenal ---
Tenés: Espada
Tenés: Arco
Tenés: Hacha
[0] Espada → daño 25
[1] Arco → daño 15
[2] Hacha → daño 40
```

🗨 Hay dos formas de recorrer: `for arma in armas` te da directo cada valor ("Espada" , "Arco" ...). `for i in range(armas.size())` te da el **número de posición** (0 , 1 , 2), útil cuando necesitás el índice para leer **dos arreglos en paralelo** al mismo tiempo.

⚠ **Ojo (de la clase):** en Godot los `for` **no** se usan para la lógica en tiempo real (para eso está `_process()`). Sirven para **inicializar**, generar contenido y recorrer datos — justo lo que hicimos acá.

✅ **Punto de control 4:** imprimís listas con `for` , con y sin índice.

🧰 Parte 5 — Funciones: escribir una vez, usar mil

Concepto: crear funciones, pasarles **parámetros** y devolver un valor con `return` .

```
extends Node

var vida = 100    # variable "global" del script: la ven todas las funciones

func _ready():
    mostrar_estado()
    recibir_dano(30)    # le pasamos un parámetro
    recibir_dano(50)
    if esta_vivo():      # usamos lo que devuelve
        print("Aria sigue en pie")
    else:
        print("Aria cayó")

# Recibe un parámetro y modifica la vida
func recibir_dano(cantidad):
    vida = vida - cantidad
    print("Recibí " + str(cantidad) + " de daño. Vida: " + str(vida))

# Devuelve un valor (true o false)
func esta_vivo():
    return vida > 0

func mostrar_estado():
    print("=== Vida actual: " + str(vida) + " ===")
```

Output:

```
=== Vida actual: 100 ===
Recibí 30 de daño. Vida: 70
Recibí 50 de daño. Vida: 20
Aria sigue en pie
```

🧠 Anatomía:

- `func recibir_dano(cantidad):` → **definición**. `cantidad` es el **parámetro** (el dato que entra).
- `recibir_dano(30)` → **llamado**. Los `()` son **obligatorios**.
- `return vida > 0` → **devuelve** un resultado (acá `true` o `false`) que podés usar en un `if`.

Ojo: `vida` está declarada **afuera** de las funciones (arriba de todo). Por eso **todas** la pueden leer y modificar. Es el "estado" del juego.

✅ **Punto de control 5:** definís funciones con parámetro y con `return`, y las llamás.

🎮 Parte 6 — Input: que reaccione a las teclas

Concepto: detectar input con `Input.is_action_just_pressed(...)` dentro de `_process()`.

Hasta ahora todo corría solo en `_ready()`. Para **reaccionar al jugador** necesitamos `_process()`, que corre en **cada frame** (~60 por segundo).

```
extends Node

func _ready():
    print("Presioná: ENTER (atacar), ← →, o ↑")

func _process(delta):
    if Input.is_action_just_pressed("ui_accept"):    # Enter / Espacio
        print("¡Ataque!")
    if Input.is_action_just_pressed("ui_right"):     # flecha →
        print("Elegiste el arma de la derecha")
    if Input.is_action_just_pressed("ui_left"):      # flecha ←
        print("Elegiste el arma de la izquierda")
    if Input.is_action_just_pressed("ui_up"):        # flecha ↑
        print("Tomaste una poción")
```

Apretá **F6**. **Importante:** hacé **clic sobre la ventana del juego** para que reciba las teclas; los mensajes aparecen en el panel **Output** del editor.

`is_action_just_pressed` se dispara **una sola vez** por pulsación (ideal para menús y ataques). `is_action_pressed` es `true` **mientras** la tenés apretada (ideal para movimiento continuo). Las acciones `ui_accept`, `ui_left`, `ui_right`, `ui_up`, `ui_down`, `ui_cancel` **ya vienen** definidas; podés crear las tuyas en `Project → Project Settings → Input Map`.

✓ **Punto de control 6:** cada tecla imprime su mensaje en Output.

🔧 **Las teclas no hacen nada**

▼ **Abrí para ver soluciones**

- ¿Hiciste **clic en la ventana del juego**? Si el foco está en el editor, no llegan las teclas.
- El código de input va en `_process(delta)`, no en `_ready()`.
- Revisá que escribiste `ui_accept` (con guion bajo), entre comillas.

🏆 **Parte 7 — Todo junto: *La Cripta del Golem***

Ahora combinamos **las 6 piezas** en un solo juego. Borrá todo el script y escribí esto. Léelo entendiendo **qué parte usa qué concepto** (están marcados con comentarios):

```

extends Node

# ---- VARIABLES: estado del juego (Parte 1) ----
var nombre = "Aria"
var vida = 100
var pociones = 2
var terminado = false

# ---- ARREGLOS en paralelo (Parte 2) ----
var armas = ["Espada", "Arco", "Hacha"]
var danos = [25, 15, 40]
var arma_actual = 0          # índice del arma elegida

# ---- Enemigo ----
var enemigo = "Golem de piedra"
var vida_enemigo = 120

func _ready():
    print("=== LA CRIPTA DEL GOLEM ===")
    print(nombre + " se enfrenta a un " + enemigo + "!")
    mostrar_inventario()      # usa un FOR (Parte 4)
    print("Controles: ← → cambiar arma | ENTER atacar | ↑ poción")
    mostrar_estado()

func _process(delta):
    # INPUT (Parte 6). Si el juego terminó, no hacemos nada.
    if terminado:
        return

    if Input.is_action_just_pressed("ui_right"):
        arma_actual = arma_actual + 1
        if arma_actual >= armas.size(): # si me pasé, vuelvo al principio
            arma_actual = 0
        print("Arma seleccionada: " + armas[arma_actual])

    if Input.is_action_just_pressed("ui_left"):
        arma_actual = arma_actual - 1
        if arma_actual < 0:
            arma_actual = armas.size() - 1
        print("Arma seleccionada: " + armas[arma_actual])

    if Input.is_action_just_pressed("ui_accept"):
        atacar()

    if Input.is_action_just_pressed("ui_up"):
        curar()

# ---- FUNCIONES (Parte 5) ----

func mostrar_inventario():

```

```

print("Tu arsenal:")
for i in range(armas.size()):          # FOR con índice
    print("  [" + str(i) + "] " + armas[i] + " (daño " + str(danos[i]) + ")")

func mostrar_estado():
    print("---")
    print(nombre + ": " + str(vida) + " HP   |   " + enemigo + ": " + str(vida_enemigo) + " HP
    | Pociones: " + str(pociones))
    print("---")

func atacar():
    var dano = danos[arma_actual]      # leo el arreglo por índice
    print(nombre + " ataca con " + armas[arma_actual] + " (" + str(dano) + " de daño)")
    vida_enemigo = vida_enemigo - dano

    if vida_enemigo <= 0:              # CONDICIONAL (Parte 3)
        print("🏆 ¡Derrotaste al " + enemigo + "! GANASTE.")
        terminado = true
        return

    enemigo_contraataca()
    mostrar_estado()

func enemigo_contraataca():
    var golpe = 20
    print("El " + enemigo + " contraataca: -" + str(golpe) + " HP")
    vida = vida - golpe
    if vida <= 0:
        print("💀 " + nombre + " cayó. GAME OVER.")
        terminado = true

func curar():
    if pociones <= 0:
        print("No te quedan pociones.")
        return
    pociones = pociones - 1
    vida = vida + 30
    print(nombre + " bebe una poción: +30 HP")
    mostrar_estado()

```

Cómo se juega: apretá **F6**, hacé **click en la ventana del juego**, y usá ← → para elegir arma, **ENTER** para atacar y ↑ para curarte. Mirá el combate desarrollarse en el panel **Output**.

✅ **Punto de control 7 (final):** podés ganarle al Golem eligiendo el Hacha y atacando, o perder si te descuidás. Todo se ve en Output.

🚫 **Errores comunes al armar el juego final**

▼ Abrió para ver soluciones

- “Invalid index / índice fuera de rango”: `armas` y `danos` tienen que tener **la misma cantidad** de elementos.
- **Indentación mezclada**: elegí espacios o tabs para todo el archivo, no los combines.
- **Se repite el ataque sin parar**: usá `is_action_just_pressed` (una vez por pulsación), no `is_action_pressed`.
- **No pasa nada al apretar teclas**: clic en la ventana del juego para darle foco.

Entrega

Entregá **una** de estas opciones (según indique el/la docente):

1. El archivo `juego.gd` (el script final), o
2. La **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`), o
3. Una **captura del panel Output** mostrando una partida (un ataque, una curación y el final).

Nombre del archivo: `tp2-crypta-ApellidoNombre.zip` (o `.gd`).

✓ Checklist de autoevaluación

- ☐ Creaste un `Node` con un script adjunto y ves salida en **Output**.
- ☐ Usás **variables** de al menos 3 tipos distintos (`int` , `String` , `bool` ...).
- ☐ Usás un **arreglo** y accedés a un elemento por **índice**.
- ☐ Usás `if` / `elif` / `else` para decidir un resultado.
- ☐ Usás un `for` para recorrer el arsenal.
- ☐ Definiste al menos **2 funciones** (una con **parámetro**, una con `return`).
- ☐ El juego reacciona a **teclas** con `Input.is_action_just_pressed` .
- ☐ Se puede **ganar** y **perder** el combate.

Extra (opcional, para los que quieran más)

- **Daño variable (azar)**: hacé que cada golpe varíe un poco. `randi_range(min, max)` devuelve un entero al azar:

```
var dano = danos[arma_actual] + randi_range(-5, 5)
```

- **Segundo enemigo:** guardá los enemigos en arreglos en paralelo (`enemigos = [...]` , `vidas_enemigo = [...]`) y pasá al siguiente con un índice cuando cae uno. Recorrellos con un `for` .
- **Función `esta_vivo()`** que devuelva `vida > 0` y usala en los `if` en vez de comparar a mano.
- **Estadísticas al final:** contá cuántos turnos duró el combate con una variable `turnos` que suba en cada ataque, e imprimila al ganar/perder.
- **Otro tema, misma estructura:** este mismo esqueleto (variables + arreglos + `if` + `for` + funciones + input) sirve para una **aventura de texto** (elegís caminos con las flechas), un **selector de pociones**, o un **gestor de recursos** tipo estrategia. Cambiás el tema, no la lógica.

Recursos

- Crear tu primer script (pasos y capturas de este TP): [docs.godotengine.org/es/4.x — Creating your first script](https://docs.godotengine.org/es/4.x/Creating_your_first_script)
- Fundamentos de GDScript: [GDScript basics](#)
- Arreglos (`Array`): [referencia de la clase Array](#)

Las capturas de este documento provienen de la **documentación oficial de Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.